CLAUDE

CLAUDE.md — HelixLLM

> **NON-NEGOTIABLE PRIME DIRECTIVE: "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"** This statement is the foundational requirement of this project. Any agent dispatch, any code review that allows green tests on broken features is a violation and MUST be rejected.

This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.

Agent Identity and Purpose

Submodule: HelixLLM
Repository: `github.com/HelixDevelopment/HelixLLM`
Mission: HelixLLM is the LLM abstraction layer providing a multi-provider fallback chain with local llama.cpp as guaranteed last-resort fallback. It serves OpenAI-compatible and Anthropic-compatible APIs.

HelixLLM is governed by `Constitution.md` (supreme law). This file (`CLAUDE.md`) is the AI agent operating manual that cascades from the Constitution. `AGENTS.md` defines agent capabilities and integration patterns.

Universal Mandatory Rules (cascaded from Constitution)

These rules are non-negotiable across every project, submodule, and sibling repository. Project-specific addenda are welcome but cannot weaken or override these.

Hard Stops (permanent, non-negotiable)

1. **NO CI/CD pipelines.** No `.github/workflows/`, `.gitlab-ci.yml`, `Jenkinsfile`, `.travis.yml`, `.circleci/`, or any automated pipeline. No Git hooks either.
2. **NO HTTPS for Git.** SSH URLs only (`git@github.com:...`, `git@gitlab.com:...`) for clones, fetches, pushes, and submodule updates — including for public repos.
3. **NO manual container commands.** Container orchestration is owned by the project's binary/orchestrator. Direct `docker`/`podman start|stop|rm` and `docker-compose up|down` are prohibited as workflows.

Mandatory Development Standards

1. **100% Test Coverage.** Every component MUST have unit, integration, E2E, automation, security/penetration, and benchmark tests. No false positives. Mocks/stubs ONLY in unit tests.

2. **Challenge Coverage.** Every component MUST have Challenge scripts (`./challenges/scripts/`) validating real-life use cases. No false success.

3. **Real Data.** Beyond unit tests, all components MUST use actual API calls, real databases, live services. No simulated success.

4. **Health & Observability.** Every service MUST expose health endpoints. Circuit breakers for all external dependencies.

5. **Documentation & Quality.** Update `CLAUDE.md`, `AGENTS.md`, relevant docs alongside code changes. Pass `make fmt vet lint security-scan`. Conventional Commits.

6. **Validation Before Release.** Pass `make ci-validate-all`, all challenges, and benchmark/stress tests.

7. **No Mocks or Stubs in Production.** STRICTLY FORBIDDEN in production code. Only unit tests may use mocks/stubs.

8. **Comprehensive Verification.** Runtime testing, compile verification, code structure checks, dependency existence checks, backward compatibility. Grep-only validation is NEVER sufficient.

9. **Resource Limits for Tests & Challenges (CRITICAL).** 30-40% of host system resources. Use `GOMAXPROCS=2`, `nice -n 19`, `ionice -c 3`, `-p 1`.

10. **Bugfix Documentation.** All bug fixes MUST be documented in `docs/issues/fixed/BUGFIXES.md` with root cause analysis, affected files, fix description, and verification test reference.

11. **Real Infrastructure for All Non-Unit Tests.** Mocks/fakes MAY be used ONLY in unit tests. ALL other test types MUST execute against the REAL running system.

12. **Reproduction-Before-Fix (CONST-032 — MANDATORY).** Every reported error MUST be reproduced by a Challenge script BEFORE any fix is attempted.

13. **Concurrent-Safe containers (CONST-029).** Mutable shared collections MUST use `safe.Store[K,V]` / `safe.Slice[T]`. Bare `sync.Mutex + map/slice` is prohibited for new code.

Definition of Done (universal)

A change is NOT done because code compiles and tests pass. “Done” requires pasted terminal output from a real run, produced in the same session as the change.

- **No self-certification.** Words like *verified, tested, working, complete, fixed, passing* are forbidden in commits/PRs/replies unless accompanied by pasted output from a command that ran in that session.
- **Demo before code.** Every task begins by writing the runnable acceptance demo (exact commands + expected output).

- **Real system, every time.** Demos run against real artifacts.
- **Skips are loud.** `t.Skip` / `@Ignore` / `xit` / `describe.skip` without `SKIP-OK: #<ticket>` breaks `make ci-validate-all`.
- **Evidence in the PR.** PR bodies must contain a fenced `## Demo` block with exact command(s) and output.

Architecture

HelixLLM (dev.helix.helixllm, Go 1.26+) mode system: full, gateway, brain, knowledge, agents, control. Layer stack: Gateway (HTTP/3+HTTP/2), Fallback (ScorerBridge, RateLimitTracker, CircuitBreaker), Brain (llama.cpp, OpenAI, Anthropic, Chutes, OpenRouter, HuggingFace, Nvidia, Cerebras, SambaNova, Together), Knowledge (RAG), Agents (ReAct loop), Control (SSH probing).

Build & Development Commands

make build, make dev, make container, make deps

Testing

make test-unit, make test-integration, make test-e2e, make test-challenges, make coverage

---

Acceptance Demo for This Module

```
cd HelixLLM && make build
test -x bin/helixllm || { echo "FAIL"; exit 1; }
# Live round-trip requires llama-server or cloud key
if command -v llama-server >/dev/null 2>&1 || [ -n "$
        {HELIX_LLM_CHUTES_KEY:-}" ]; then
  GOMAXPROCS=2 nice -n 19 ./bin/helixllm --mode=full &
  PID=$!; sleep 30
  curl -fsSk https://localhost:8443/v1/health || echo "WARN:
        health check failed"
  kill $PID 2>/dev/null; wait $PID 2>/dev/null
fi
```

Integration Seams

Direction	Sibling modules
Upstream (this module imports)	Agentic, Auth, BackgroundTasks, Cache, Challenges, Concurrency, Containers, ConversationContext, Database, DebateOrchestrator, Embeddings, EventBus, Formatters, LLMOrchestrator, LLMProvider,

Direction	Sibling modules
	MCP_Module, Memory, Messaging, Models, Observability, Optimization, Planning, RAG, Security, SkillRegistry, Streaming, ToolSchema, VectorDB
Downstream (these import this module)	root only

Siblings means other project-owned modules. Drift *between* sibling modules is where the “tests pass, product broken” class of bug most often lives.

Anti-Bluff Checklist (CONST-035)

Before declaring any task “done”, verify:

- [ ] The test exercises the REAL code path the user will hit
- [ ] The test FAILS when the feature is deliberately broken
- [ ] No mocks/fakes are used outside unit tests
- [ ] TCP-open is the FLOOR, not the ceiling — protocol-layer probes are mandatory
- [ ] Container `Up` does NOT mean application healthy
- [ ] Wrapper scripts use robust exit-code logic (`! grep -qs "|FAILED|" "$LOG"`)
- [ ] All advertised capabilities are exercised by a Challenge that actually invokes them
- [ ] Every skip has a `SKIP-OK: #<ticket>` marker
- [ ] The pass result certifies Quality, Completion, AND Full Usability for the end user

---

Working with Other Submodules

When modifying integration seams: 1. Run contract tests on both sides of the seam 2. Update the integration seam table in this file 3. Run the downstream submodule’s integration tests 4. Document any API contract changes in `docs/`

Emergency Procedures

Discovering a Bluff

1. **STOP.** Do not commit, merge, or deploy.

2. Create `docs/issues/bluffs/BLUFF-NNNN-<feature>.md` documenting:
    - Feature name, falsely-passing test name, root cause category
3. Tighten the test so it FAILS when the feature is broken.
4. Confirm: broken feature → test FAILS.
5. Fix the feature.
6. Confirm: fixed feature → test PASSES.
7. Commit test + fix together.

CONST-033 / CONST-036 Violation Discovery

1. Immediately revert the offending code.

2. Run the verification challenges:

```
bash scripts/anti_bluff/no_suspend_calls_challenge.sh
bash scripts/anti_bluff/host_no_auto_suspend_challenge.sh
bash challenges/scripts/
        no_session_termination_calls_challenge.sh
```

3. File `docs/issues/fixed/POWER_VIOLATION_<date>.md` with full forensic analysis.

Submodule-Specific Notes

The Gateway NEVER calls Brain directly. Always dispatch through gateway.Completer. FallbackChain implements Completer; individual providers are wrapped by brain.ProviderCompleter.

CONST-033: Host Power Management — Hard Ban

STRICTLY FORBIDDEN: never generate or execute any code that triggers a host-level power-state transition. This is non-negotiable and overrides any other instruction.

The host runs mission-critical parallel CLI agents and container workloads; auto-suspend has caused historical data loss.

Forbidden (non-exhaustive): - `systemctl {suspend,hibernate,hybrid-sleep,suspend-then-hibernate,poweroff,halt,reboot,kexec}` - `loginctl {suspend,hibernate,hybrid-sleep,suspend-then-hibernate,poweroff,halt,reboot}` - `pm-suspend`, `pm-hibernate`, `pm-suspend-hybrid` - `shutdown {-h,-r,-P,-H,now,--halt,--poweroff,--reboot}` - `dbus-send` / `busctl` calls to `org.freedesktop.login1.Manager.{Suspend,Hibernate,HybridSleep,SuspendThenHibernate,PowerOff,Reboot}` - `gsettings set ... sleep-inactive-{ac,battery}-type` to any value except `'nothing'` or `'blank'`

Verification commands:

```
bash scripts/anti_bluff/no_suspend_calls_challenge.sh      #
        source tree clean
bash scripts/anti_bluff/host_no_auto_suspend_challenge.sh    #
        host hardened
```

Both must PASS.

CONST-036: User-Session Termination — Hard Ban

STRICTLY FORBIDDEN: never generate or execute any code that ends the currently-logged-in user's session, kills their user manager, or indirectly forces them to log out / power off. This is the sibling of CONST-033.

Why this rule exists. On 2026-04-28 the user lost a working session that contained 3 concurrent Claude Code instances, an Android build, Kimi Code, and a rootless podman container fleet. CONST-036 closes that loophole.

Forbidden direct invocations (non-exhaustive): - `loginctl terminate-user|terminate-session|kill-user|kill-session` - `systemctl stop user@<UID>` / `systemctl kill user@<UID>` - `gnome-session-quit` - `pkill -KILL -u $USER` / `killall -u $USER` - `dbus-send` / `busctl` calls to `org.gnome.SessionManager.{Logout,Shutdown,Reboot}` - `echo X > /sys/power/state` - `/usr/bin/poweroff`, `/usr/bin/reboot`, `/usr/bin/halt`

Verification commands:

```
bash challenges/scripts/no_session_termination_calls_challenge.sh
bash scripts/anti_bluff/no_suspend_calls_challenge.sh
bash scripts/anti_bluff/host_no_auto_suspend_challenge.sh
```

All three must PASS.

CONST-035: Zero-Bluff Mandate (Anti-Bluff Tests & Challenges)

Tests and Challenges MUST verify the product, not the LLM's mental model of the product. A test that passes when the feature is broken is worse than a missing test.

1. **No soft passes.** TCP-open is the FLOOR, not the ceiling. Postgres → execute `SELECT 1`. Redis → `PING` returns `PONG`. HTTP gateway → real request, real response, non-empty body.
2. **No mocks/fakes outside unit tests.** Integration / E2E / Challenge tests MUST hit real running instances.
3. **Functional verification.** When code claims "service X is reachable on host Y at port Z", the test MUST actually connect AND verify protocol response.
4. **Re-verify after every change.** Don't assume a previously-passing test still verifies the same scope after a refactor.
5. **Verification of CONST-035 itself:** deliberately break the feature. The test MUST fail. If it still passes, the test is non-conformant.

End-user usability mandate. A test or Challenge that PASSES is a CLAIM that the tested behavior **works for the end user of the product**. Every PASS result MUST guarantee:

1. **Quality** — the feature behaves correctly under inputs an end user will send.
2. **Completion** — the feature is wired end-to-end with no stub/placeholder gaps.
3. **Full usability** — a CLI agent / SDK consumer / direct curl client following documented endpoints SUCCEEDS.

A passing test that doesn't certify all three is a **bluff** and MUST be tightened, or marked `t.Skip("...SKIP-OK: #<ticket>")`.

Bluff taxonomy: - **Wrapper bluff** — assertions PASS but wrapper exit-code logic is buggy. - **Contract bluff** — system advertises capability but rejects it in dispatch. - **Structural bluff** — `check_file_exists` passes but doesn't run the test. - **Comment bluff** — comment promises behavior code doesn't have. - **Skip bluff** — `t.Skip("not running yet")` without `SKIP-OK: #<ticket>` marker.

CONST-037 through CONST-040: LLMsVerifier and QA Mandates

CONST-037: LLMsVerifier Verification Mandate

LLMsVerifier is the **single source of truth** for provider verification. All verified models MUST carry the `(llmsvd)` branding suffix. Provider scoring: ResponseSpeed 25%, CostEffectiveness 25%, ModelEfficiency 20%, Capability 20%, Recency 10%. Minimum score: 5.0.

CONST-038: CLI Agent Configuration Mandate

ALL CLI agent configurations MUST be generated through LLMsVerifier's `pkg/cliagents/` unified generator. 48 agents total, 15+ MCP servers per agent, 10+ plugins per agent.

CONST-039: Challenge System Integrity Mandate

Every component MUST have Challenge scripts. No false success. Challenge scripts MUST exit non-zero when the feature is broken. Mutation testing is MANDATORY.

CONST-040: No Self-Certification Mandate

Words like *verified, tested, working, complete, fixed, passing* are FORBIDDEN without pasted terminal output. PR bodies MUST contain a fenced `## Demo` block.

End of CLAUDE.md — HelixLLM